

Guided Lab 3: Cubic Spline

Due March 6, 2026 by 6pm

Please show all your work and include any code you create along with your submission. Be sure to acknowledge any substantial help you receive. This includes collaboration with others, references and online resources.

In this guided lab, you will code your own cubic spline interpolator that uses natural boundary conditions. Bradie's *A Friendly Introduction to Numerical Analysis*, pages 393–396, explains how to set up the equations for determining the coefficients for a cubic spline with not-a-knot boundary conditions. A description of natural boundary conditions is found below Exercise 13 on page 404.

Please copy the template `cubic_spline_template.m` and rename it as `cubic_spline.m` to use as a basis for this project. You can also translate the template into another programming language of your choosing if you prefer. You should be able to find analogous commands and packages described below for python, C, or other common languages for scientific computing.

- Questions 1-3 ask you to sequentially add to the template to create a cubic spline interpolator.
- Question 4 asks you to interpret and explain code already given.
- Questions 5-6 ask you to run your interpolator with two different sets of data. You may want to create a script that includes both data sets, from which you call `cubic_spline.m` for each interpolation.
- Questions 7 and asks you to use your interpolator on a data set of your choosing.

In questions 1-3 you are asked to use the provided template to create code that calculates the coefficients for natural cubic splines interpolants.

1. Assume you are given $n + 1$ data points in the form of $x = [x_1, x_2, \dots, x_{n+1}]^T$ and $y = [y_1, y_2, \dots, y_{n+1}]^T$. Write a function or subroutine that takes x, y and produces the matrix A and the right-hand side vector $\mathbf{rvec}$, as seen in the text, with indexing augmented by 1 (since matlab starts with 1 and not 0). Note that, for example, mathematical indexing notation h_j corresponds to the matlab syntax $\mathbf{h}(j)$.

$$A = \begin{bmatrix} 1 & 0 & 0 & \dots & \dots & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & 0 & \dots & 0 \\ 0 & h_2 & 2(h_2 + h_3) & h_3 & 0 \dots & 0 \\ \vdots & 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & 0 & \ddots & h_{n-1} & 2(h_{n-1} + h_n) & h_n \\ 0 & \dots & \dots & \dots & 0 & 1 \end{bmatrix} \text{ and}$$

$$\vec{r} = \begin{bmatrix} 0 \\ 3 \left(\frac{a_3 - a_2}{h_2} - \frac{a_2 - a_1}{h_1} \right) \\ 3 \left(\frac{a_4 - a_3}{h_3} - \frac{a_3 - a_2}{h_2} \right) \\ \vdots \\ 3 \left(\frac{a_{n+1} - a_n}{h_n} - \frac{a_n - a_{n-1}}{h_{n-1}} \right) \\ 0 \end{bmatrix} \quad \text{with } a_j = y_j \text{ for } j = 1 \dots n + 1.$$

You might find the following representations useful for coding:

- (a) $h(j) \longleftrightarrow x_{j+1} - x_j$
- (b) $A(j, j-1) \longleftrightarrow h_{j-1}$
- (c) $A(j, j) \longleftrightarrow 2(h_{j-1} + h_j)$
- (d) $A(j, j+1) \longleftrightarrow h_j$

Note also that the natural boundary conditions give that $A(1, 1) = A(n+1, n+1) = 1$, and $c_1 = c_{n+1} = 0$.

2. Add to the function from question 1 to solve $A\vec{c} = \vec{r}$ for $\vec{c}$ using the command `c=A\rvec`. Alternatively, you could write your own linear solver. Since A is tridiagonal, you could use the modified version of the Crout algorithm described on pages 217-219 (and appendix B) of Bradie which takes only $\mathcal{O}(n)$ operations.

3. Add to the function from questions 1-2 to create vectors of a_j 's, b_j 's, and d_j 's, using the following formulas

- (a) $a(j) \longleftrightarrow y_j$ for $j = 1 \dots n + 1$
- (b) $d(j) \longleftrightarrow \frac{c_{j+1} - c_j}{3h_j}$ for $j = 1 \dots n$
- (c) $b(j) \longleftrightarrow \frac{a_{j+1} - a_j}{h_j} - \frac{1}{3}(2c_j + c_{j+1})h_j$ for $j = 1 \dots n$

Now you have calculated values for all a_j, b_j, c_j, d_j . The next question asks you to interpret existing code.

4. There is code already in `cubic_spline_template.m` under "Question 4" for using the coefficients of the piecewise spline to evaluate the spline function at any input x value or values. For this question describe what is happening in each line. You will need to look at the help files for `mkpp` and `ppval`. Do not edit this code!

In questions 5-6 you will be validating your code with an example from the Bradie excerpt, and then using it to solve a new interpolation problem.

5. Test your code using the data set on page 397 of the Bradie text, “Mean Activity Coefficient of Silver Nitrate.” Evaluate the natural cubic spline interpolant at 0.032. You should get a result close to 0.828. (It may not be exact, because 0.828 was found with not-a-knot boundary conditions.) Show the output from your test.
6. It is suspected that the high amounts of tannin in mature oak leaves inhibit the growth of the winter moth larvae that damage these trees in certain years. The following table lists the average weight of a sample of larvae at times in the first 28 days after birth. The first data set is based on larvae taken from mature leaves while the second is based on larvae taken from young leaves

Table 1: Average weights of larvae, from Feeny, P. (1970). *Ecology*, **51**(4), 565–581.

Days After birth	0	6	10	13	17	20	28
avg. weight, <i>mg</i> (from young leaves)	7	18	45	40	32	30.5	30
avg. weight, <i>mg</i> (from mature leaves)	7	16	19	15	12	10.5	10

- (a) Estimate the mean weight of a larva three days after birth using your interpolator. Use Matlab’s built in `spline()` command to compare your estimate to one from a cubic spline with not-a-knot boundary conditions. What do you notice?
- (b) Use your interpolator to estimate the the largest mean weights for larvae taken from mature leaves and young leaves. In each case, does this peak occur before or after the data point at 10 days? Is the data consistent with the hypothesis that tannins in mature leaves inhibit growth, why or why not?

In questions 7 and 8, you will use your code to interpolate data of your own choosing.

7. Select a data set of interest to you. It should have at least 10 data points (x_i, y_i) , which can be plotted in the plane. Describe the data, where it came from, and why it is interesting. Using a `for` loop, interpolate the data at 100 or more equally spaced points and store the values in an array. Plot the estimated values with a line connecting them. Add a plot of the original data points with circles as markers, but no connecting lines.
8. Repeat question 7, except use only every other data points to construct your interpolant. Compare the estimated values to the actual data at the points you did not use in your interpolation. What happens if you use every 3rd data point instead?